

DOCUMENTATION

ASSIGNMENT 3

STUDENT NAME: TANASE LUISA ELENA
GROUP: 30422

CONTENTS

1. Assignment Objective	1
2. Problem Analysis, Modeling, Scenarios, Use Cases	1
3. Design	9
4. Implementation	11
5. Conclusions	27
6. Bibliography	27

1. Assignment Objective

Main objective:

- The main objective is to design and implement an application for managing the client orders for a warehouse

Sub-objectives:

- The user can make CRUD operations on clients, products and can create orders.
- The user can select a client and the products for the order and also select the quantity for each product in the order. If the selected quantity is greater than the stock for that product, a warning is displayed. After creating an order the stock is decremented.
- After each successfully placed order, a bill is generated.

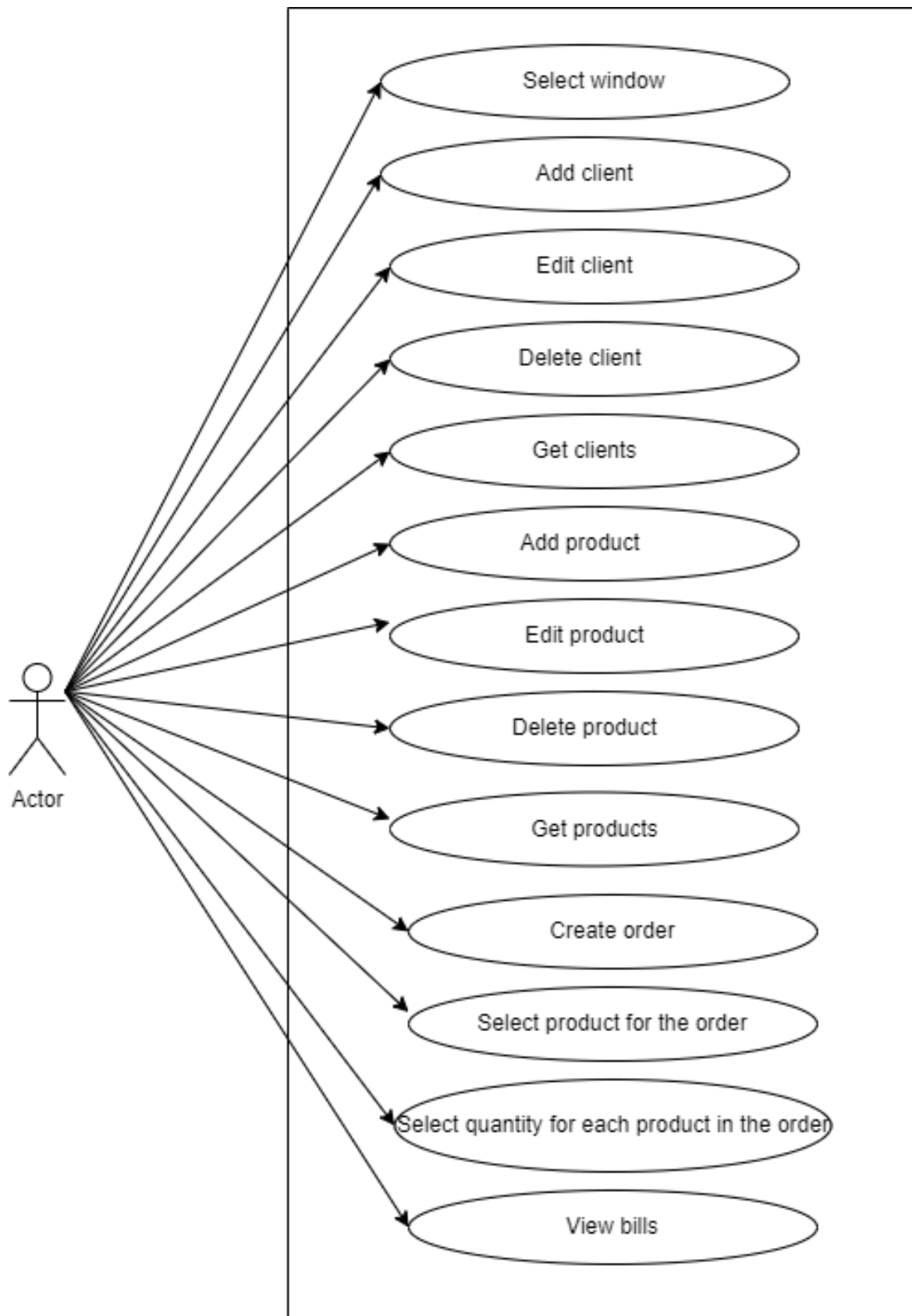
2. Problem Analysis, Modeling, Scenarios, Use Cases

Functional Requirements:

- The application should allow the user to select what window to display next: operations for clients, products, create a new order or view the bills.
- After that, each specific window provides intuitive components to allow the user to perform the desired CRUD operations on the database.

Non-Functional Requirements:

- The application should be intuitive and easy to use;
- The application should provide a quick response to user inputs;
- The application should handle edge cases gracefully and display friendly error messages;



Use Case: select window

Primary Actor: employee

Main Success Scenario:

1. The employee clicks one of the buttons from the main window: clients, products, create new order or view bills.
2. If the employee chose “clients” then it will be directed to a window where it could insert, edit, delete or get all clients in a table.
3. If the employee chose “products” then it will be directed to a window where it could insert, edit, delete or get all products in a table.
4. If the employee chose “create new order” then it will be directed to a window where it could select the client for which the new order is and then it could select the products and their corresponding quantity for that order.
5. If the employee chose “view bills” then it will be directed to a window where it could view all the bills in a table.

Use Case: add product

Primary Actor: employee

Main Success Scenario:

1. The employee selects the option to add a new product
 2. The application will display a form in which the product details should be inserted
 3. The employee inserts the name of the product, its price and current stock
 4. The employee clicks on the “Add” button
 5. The application stores the product data in the database and displays an acknowledge message
- Alternative Sequence: Invalid values for the product’s data
- The user inserts a negative value for the stock of the product
 - The application displays an error message and requests the user to insert a valid stock
 - The scenario returns to step 3

Use Case: edit product

Primary Actor: employee

Main Success Scenario:

1. The employee selects the option to edit a product
2. The application will display a form in which the updated product details should be inserted
3. The employee inserts the new data for the product to be updated.
4. The employee clicks on the “Edit” button
5. The application stores the product data in the database and displays an acknowledge message

Alternative Sequence: Invalid values for the product's data

- The user inserts a negative value for the stock of the product
- The application displays an error message and requests the user to insert a valid stock
- The scenario returns to step 3

Use Case: delete product

Primary Actor: employee

Main Success Scenario:

1. The employee selects the option to delete a product
2. The application will display a form in which the product details should be inserted
3. The employee inserts the id of the product to be deleted
4. The employee clicks on the "Delete" button
5. The application displays an acknowledge message

Use Case: get products

Primary Actor: employee

Main Success Scenario:

1. The employee selects the option to get all products
2. The application will display a table with all the products in the database and their properties.

Use Case: add client

Primary Actor: employee

Main Success Scenario:

1. The employee selects the option to add a new client
2. The application will display a form in which the client details should be inserted
3. The employee inserts the name of the client and its email.
4. The employee clicks on the "Insert" button
5. The application stores the client data in the database and displays an acknowledge message

Alternative Sequence: Invalid values for the client's data

- The user inserts an invalid email
- The application displays an error message and requests the user to insert a valid stock
- The scenario returns to step 3

Use Case: edit client

Primary Actor: employee

Main Success Scenario:

1. The employee selects the option to edit a client
2. The application will display a form in which the updated client details should be inserted
3. The employee inserts the new data for the client to be updated.
4. The employee clicks on the “Edit” button
5. The application stores the client data in the database and displays an acknowledge message

Alternative Sequence: Invalid values for the client’s data

- The user inserts an invalid email
- The application displays an error message and requests the user to insert a valid stock
- The scenario returns to step 3

Use Case: delete client

Primary Actor: employee

Main Success Scenario:

1. The employee selects the option to delete a client
2. The application will display a form in which the client details should be inserted
3. The employee inserts the id of the client to be deleted
4. The employee clicks on the “Delete” button
5. The application displays an acknowledge message

Use Case: get clients

Primary Actor: employee

Main Success Scenario:

1. The employee selects the option to get all clients
2. The application will display a table with all the clients in the database and their properties.

Use Case: create order

Primary Actor: employee

Main Success Scenario:

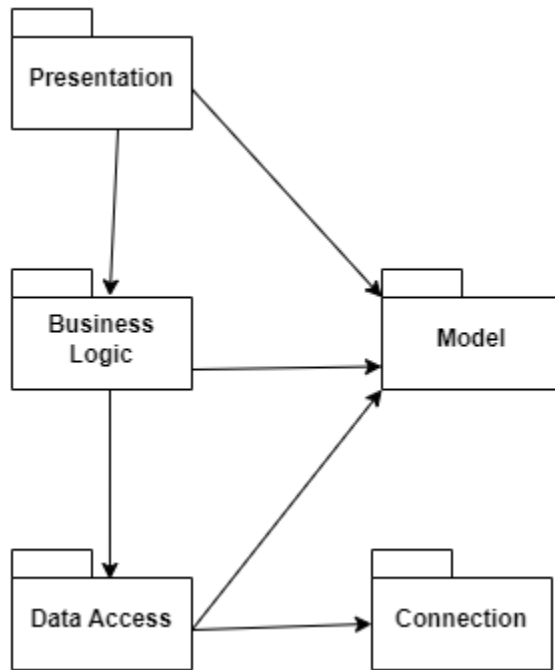
1. The employee selects the option to create a new order
2. The application will display a combo box for selecting the client for which the new order is
3. The employee inserts the name of the product, its price and
4. The employee clicks on the name of the client in order to select it for the new order.
5. The application displays a radio button corresponding to the products in the database.
6. The employee clicks on the desired product for the order and a pop-up window will allow the employee to select the desired quantity for that product in the order.
7. The employee can select multiple products for an order.
8. The employee clicks the "create order" button.

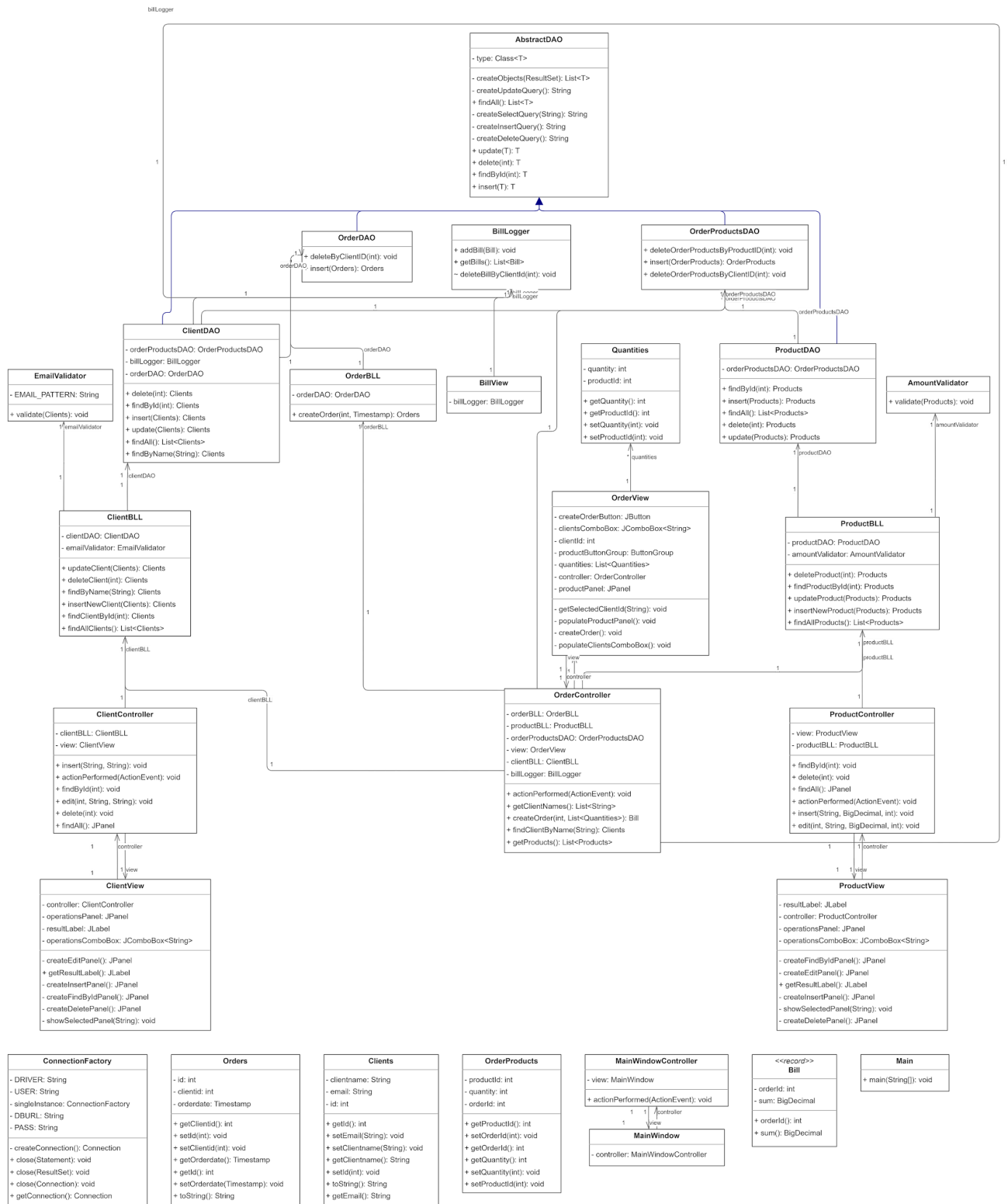
Alternative Sequence: Invalid values for the product's quantity

- The user inserts a negative value for the quantity of the product
 - The user inserts a decimal value for the quantity of the product
 - The user inserts a value for the quantity of the product greater than the product stock
-
- The application displays an error message and requests the user to insert a valid quantity
 - The scenario returns to step 6

3. Design

Package diagram





4. Implementation

Class: **MainWindow**

Description:

The **MainWindow** class represents the main window of the application. It extends the **JFrame** class to create a graphical user interface (GUI) window for the application.

Fields:

- **private MainWindowController controller**: Represents the controller responsible for handling user interactions in the main window.

Constructors:

- **public MainWindow()**: Initializes and sets up the main window with buttons for different operations.

Methods:

- **setTitle(String title)**: Sets the title of the main window.
- **setDefaultCloseOperation(int operation)**: Sets the default close operation when the user closes the window.
- **setSize(int width, int height)**: Sets the size of the main window.
- **setVisible(boolean visible)**: Sets the visibility of the main window.
- **void actionPerformed(ActionEvent e)**: Handles action events triggered by button clicks in the main window.

Class: **MainWindowController**

Description:

The **MainWindowController** class handles user actions from the main window. It implements the **ActionListener** interface to respond to menu item clicks.

Fields:

- **private MainWindow view**: Represents the main window view associated with this controller.

Constructors:

- **public MainWindowController(MainWindow view)**: Constructs a new **MainWindowController** object with the specified main window view.

Methods:

- **public void actionPerformed(ActionEvent e):** Handles action events triggered by menu item clicks in the main window.

Class: ClientView

Description:

The **ClientView** class represents the graphical user interface (GUI) for interacting with client data. It extends **JFrame** to create a window for displaying various operations related to clients.

Fields:

- **private ClientController controller:** Represents the controller associated with this view.
- **private JComboBox<String> operationsComboBox:** Represents a combo box for selecting different client operations.
- **private JPanel operationsPanel:** Represents the panel where different operation panels are displayed.
- **private JLabel resultLabel:** Represents a label for displaying operation results or messages.

Constructors:

- **public ClientView():** Constructs a new **ClientView** object, initializes the GUI components, and makes the window visible.

Methods:

- **private JPanel createFindByIdPanel():** Creates a panel for finding a client by ID.
- **private JPanel createInsertPanel():** Creates a panel for inserting a new client.
- **private JPanel createDeletePanel():** Creates a panel for deleting a client by ID.
- **private JPanel createEditPanel():** Creates a panel for editing an existing client.
- **private void showSelectedPanel(String selectedItem):** Shows the selected panel based on the operation chosen in the combo box.
- **public JLabel getResultLabel():** Returns the result label used for displaying operation results or messages.

Class: ClientController

Description:

The **ClientController** class acts as the controller for client-related operations in the GUI. It implements the **ActionListener** interface to handle user actions from the view.

Fields:

- **private ClientView view:** Represents the client view associated with this controller.
- **private ClientBLL clientBLL:** Manages business logic operations related to clients.

Constructors:

- **public ClientController(ClientView view):** Constructs a new **ClientController** object with the specified client view and initializes the **ClientBLL** for business logic operations.

Methods:

- **public void actionPerformed(ActionEvent e):** Handles action events triggered by user actions in the view (not implemented in this class).

Public Methods:

- **public void findById(int id):** Finds a client by ID and updates the view's result label.
- **public void edit(int id, String name, String email):** Updates an existing client's information and updates the view's result label.
- **public void delete(int id):** Deletes a client by ID and updates the view's result label.
- **public void insert(String name, String email):** Inserts a new client with the specified name and email and updates the view's result label.
- **public JPanel findAll():** Retrieves all clients from the data source and displays them in a table within a JPanel.

Class: ProductView

Description:

The **ProductView** class represents the graphical user interface (GUI) for managing products. It extends **JFrame** to create a window for displaying various product operations.

Fields:

- **private ProductController controller:** Represents the controller associated with this view.
- **private JComboBox<String> operationsComboBox:** Represents a combo box for selecting different product operations.
- **private JPanel operationsPanel:** Represents the panel where different operation panels are displayed.
- **private JLabel resultLabel:** Represents a label for displaying operation results or messages.

Constructors:

- **public ProductView():** Constructs a new **ProductView** object, initializes the GUI components, and makes the window visible.

Methods:

- **private JPanel createFindByIdPanel():** Creates a panel for finding a product by ID.
- **private JPanel createInsertPanel():** Creates a panel for inserting a new product.
- **private JPanel createDeletePanel():** Creates a panel for deleting a product by ID.
- **private JPanel createEditPanel():** Creates a panel for editing an existing product.
- **private void showSelectedPanel(String selectedItem):** Shows the selected panel based on the operation chosen in the combo box.
- **public JLabel getResultLabel():** Returns the result label used for displaying operation results or messages.

Class: ProductController

Description:

The **ProductController** class acts as the controller for product-related operations in the GUI. It implements the **ActionListener** interface to handle user actions from the view.

Fields:

- **private ProductView view**: Represents the product view associated with this controller.
- **private ProductBLL productBLL**: Manages business logic operations related to products.

Constructors:

- **public ProductController(ProductView view)**: Constructs a new **ProductController** object with the specified product view and initializes the **ProductBLL** for business logic operations.

Methods:

- **public void actionPerformed(ActionEvent e)**: Handles action events triggered by user actions in the view (not implemented in this class).

Public Methods:

- **public void findById(int id)**: Finds a product by ID and updates the view's result label.
- **public void edit(int id, String name, BigDecimal price, int amount)**: Updates an existing product's information and updates the view's result label.
- **public void delete(int id)**: Deletes a product by ID and updates the view's result label.
- **public void insert(String name, BigDecimal price, int amount)**: Inserts a new product with the specified name, price, and amount and updates the view's result label.
- **public JPanel findAll()**: Retrieves all products from the data source and displays them in a table within a JPanel.

Class: OrderView**Description:**

The **OrderView** class represents the graphical user interface (GUI) for managing orders. It extends **JFrame** to create a window for creating orders with clients and products.

Fields:

- **private OrderController controller**: Represents the controller associated with this view.
- **private JComboBox<String> clientsComboBox**: Represents a combo box for selecting clients.
- **private JPanel productPanel**: Represents the panel for displaying products and quantities for order creation.
- **private int clientId**: Represents the ID of the selected client.
- **private ButtonGroup productButtonGroup**: Represents the button group for product selection.
- **private List<Quantities> quantities**: Represents a list of product quantities selected for the order.
- **private JButton createOrderButton**: Represents the button to create an order.

Constructors:

- **public OrderView()**: Constructs a new **OrderView** object, initializes the GUI components, and makes the window visible.

Methods:

- **private void populateClientsComboBox():** Populates the clients combo box with client names retrieved from the controller.
- **private void getSelectedClientId(String selectedItem):** Gets the ID of the selected client based on the client name from the combo box.
- **private void populateProductPanel():** Populates the product panel with radio buttons for product selection.
- **private void createOrder():** Validates client and product selections, then creates an order using the controller.

Class: OrderController

Description:

The **OrderController** class acts as the controller for handling orders and interacting with the view and business logic layers.

Fields:

- **private OrderView view:** Represents the associated view for order management.
- **private ClientBLL clientBLL:** Manages client-related business logic operations.
- **private ProductBLL productBLL:** Manages product-related business logic operations.
- **private OrderProductsDAO orderProductsDAO:** Handles data access operations for order products.
- **private BillLogger billLogger:** Handles logging of bills.
- **private OrderBLL orderBLL:** Manages order-related business logic operations.

Constructors:

- **public OrderController(OrderView view):** Constructs an **OrderController** object with the specified view.

Methods:

- **public List<String> getClientNames():** Retrieves a list of client names from the database.
- **public Clients findClientByName(String name):** Finds a client by their name.
- **public List<Products> getProducts():** Retrieves a list of products from the database.
- **public Bill createOrder(int clientId, List<Quantities> quantities):** Creates an order for a client with specified product quantities.

Record: Bill

Description:

The **Bill** record represents information about a bill in the system, including the order ID and the total sum of the bill.

Fields:

- **private final int orderId**: Represents the ID of the order associated with the bill.
- **private final BigDecimal sum**: Represents the total sum of the bill.

Constructors:

- **public Bill(int orderId, BigDecimal sum)**: Constructs a new **Bill** object with the specified order ID and total sum.

Class: Clients

Description:

The **Clients** class represents information about a client in the system, including their ID, name, and email.

Fields:

- **private int id**: Represents the ID of the client.
- **private String clientname**: Represents the name of the client.
- **private String email**: Represents the email address of the client.

Constructors:

1. **public Clients() {}**: Default constructor.
2. **public Clients(String clientname, String email)**: Constructor to initialize a client with a name and email.
3. **public Clients(int id, String clientname, String email)**: Constructor to initialize a client with an ID, name, and email.

Methods:

- **public int getId()**: Returns the ID of the client.
- **public String getClientname()**: Returns the name of the client.
- **public String getEmail()**: Returns the email address of the client.
- **public void setId(int id)**: Sets the ID of the client.
- **public void setClientname(String clientname)**: Sets the name of the client.
- **public void setEmail(String email)**: Sets the email address of the client.
- **@Override public String toString()**: Returns a formatted string representation of the client's information.

Public Methods:

- **toString()**: Overrides the **toString()** method to provide a formatted string representation of the client's information.

Class: OrderProducts

Description:

The **OrderProducts** class represents a product in an order, containing information such as the order ID, product ID, and quantity.

Fields:

- **private int orderId**: Represents the ID of the order.
- **private int productId**: Represents the ID of the product.
- **private int quantity**: Represents the quantity of the product in the order.

Constructors:

1. **public OrderProducts() {}**: Default constructor.
2. **public OrderProducts(int orderId, int productId, int quantity)**: Constructor to initialize an order product with an order ID, product ID, and quantity.

Methods:

- **public int getOrderId()**: Returns the ID of the order.
- **public void setOrderId(int orderId)**: Sets the ID of the order.
- **public int getProductId()**: Returns the ID of the product.
- **public void setProductId(int productId)**: Sets the ID of the product.
- **public int getQuantity()**: Returns the quantity of the product.
- **public void setQuantity(int quantity)**: Sets the quantity of the product.

Class: Orders**Description:**

The **Orders** class represents an order entity, containing information such as the order ID, client ID, and order date.

Fields:

- **private int id**: Represents the ID of the order.
- **private int clientid**: Represents the ID of the client associated with the order.
- **private Timestamp orderdate**: Represents the date and time when the order was placed.

Constructors:

1. **public Orders() {}**: Default constructor.
2. **public Orders(int clientid, Timestamp orderdate)**: Constructor to initialize an order with a client ID and order date.
3. **public Orders(int id, int clientid, Timestamp orderdate)**: Constructor to initialize an order with an ID, client ID, and order date.

Methods:

- **public int getId()**: Returns the ID of the order.
- **public int getClientid()**: Returns the ID of the client associated with the order.
- **public Timestamp getOrderdate()**: Returns the order date and time.
- **public void setId(int id)**: Sets the ID of the order.
- **public void setClientid(int clientid)**: Sets the ID of the client associated with the order.
- **public void setOrderdate(Timestamp orderdate)**: Sets the order date and time.

Description:

The **Products** class represents a product entity, containing information such as the product ID, name, price, and available amount.

Fields:

- **private int id**: Represents the ID of the product.
- **private String productname**: Represents the name of the product.
- **private BigDecimal price**: Represents the price of the product.
- **private int amount**: Represents the available amount of the product.

Constructors:

1. **public Products() {}**: Default constructor.
2. **public Products(String name, BigDecimal price, int amount)**: Constructor to initialize a product with its name, price, and amount.
3. **public Products(int productid, String productname, BigDecimal price, int amount)**: Constructor to initialize a product with an ID, name, price, and amount.

Methods:

- **public int getId()**: Returns the ID of the product.
- **public String getProductname()**: Returns the name of the product.
- **public BigDecimal getPrice()**: Returns the price of the product.
- **public int getAmount()**: Returns the available amount of the product.
- **public void setId(int productid)**: Sets the ID of the product.
- **public void setProductname(String productname)**: Sets the name of the product.
- **public void setPrice(BigDecimal price)**: Sets the price of the product.
- **public void setAmount(int amount)**: Sets the available amount of the product.

Class: Quantities**Description:**

The **Quantities** class represents a pairing of a product ID and its corresponding quantity in an order.

Fields:

- **private int productId**: Represents the ID of the product.
- **private int quantity**: Represents the quantity of the product.

Constructor:

- **public Quantities(int productId, int quantity)**: Initializes a **Quantities** object with a product ID and quantity.

Methods:

- **public int getProductId()**: Returns the product ID.
- **public void setProductId(int productId)**: Sets the product ID.
- **public int getQuantity()**: Returns the quantity.
- **public void setQuantity(int quantity)**: Sets the quantity.

Class: AbstractDAO

Description:

The **AbstractDAO** class is a generic Data Access Object (DAO) designed to perform common database operations on entities. It is parameterized by the type **T**, representing the entity type to be handled by this DAO.

Constructor

- **public AbstractDAO():** Constructs an **AbstractDAO** instance, inferring the type **T** based on its subclass.

Fields

- **private final Class<T> type:** Represents the class type of the entity (**T**) managed by this DAO.

Methods

- **public List<T> findAll():** Retrieves all entities of type **T** from the database.
- **public T findById(int id):** Finds an entity by its ID in the database.
- **public T insert(T t):** Inserts a new entity into the database.
- **public T update(T t):** Updates an existing entity in the database.
- **public T delete(int id):** Deletes an entity from the database by its ID.

Private Methods

- **createSelectQuery(String field):** Creates a SELECT query based on a specified field.
- **createInsertQuery():** Creates an INSERT query for the entity type.
- **createUpdateQuery():** Creates an UPDATE query for the entity type.
- **createDeleteQuery():** Creates a DELETE query for the entity type.
- **createObjects(ResultSet resultSet):** Maps ResultSet rows to entity objects using reflection.

Database Operations

- The class uses **ConnectionFactory** to obtain database connections.
- It handles SQL queries dynamically based on the entity type and fields.
- Proper exception handling and resource management ensure reliable database operations.

Class: BillLogger

Description:

The **BillLogger** class manages the logging and retrieval of bills in the database.

Constructors

- None

Methods

- **public void addBill(Bill bill):** Adds a bill to the log table in the database.
- **public List<Bill> getBills():** Retrieves a list of all bills logged in the database.

Method Details

- **addBill:** Inserts a new bill record into the **log** table using the provided **Bill** object. It retrieves the connection from the **ConnectionFactory** class to execute the SQL INSERT command.

- **getBills:** Fetches all bill records from the **log** table and constructs a list of **Bill** objects based on the retrieved data. It returns the list of logged bills.

Class: ClientDAO

Description: The **ClientDAO** class manages database operations related to clients, including finding clients by ID or name, inserting new clients, updating existing client records, and deleting clients from the database.

Fields:

- None explicitly declared in this class.

Constructor:

- **public ClientDAO():** Constructs a new **ClientDAO** instance. Inherits from the **AbstractDAO** class.

Methods:

findById

- **Description:** Finds a client by their ID in the database.
- **Parameters:**
 - **int clientId:** The ID of the client to find.
- **Returns:** The **Clients** object representing the found client, or **null** if not found.

insert

- **Description:** Inserts a new client into the database.
- **Parameters:**
 - **Clients newClient:** The **Clients** object representing the new client to insert.
- **Returns:** The inserted client object with updated ID if applicable, or **null** on failure.

update

- **Description:** Updates an existing client in the database.
- **Parameters:**
 - **Clients clientWithUpdatedValues:** The **Clients** object representing the updated client values.
- **Returns:** The updated client object, or **null** on failure.

delete

- **Description:** Deletes a client from the database by its ID.
- **Parameters:**
 - **int id:** The ID of the client to delete.
- **Returns:** The deleted client object, or **null** on failure.

findAll

- **Description:** Retrieves a list of all clients from the database.
- **Parameters:** None
- **Returns:** A list of **Clients** objects representing all clients found.

findByName

- **Description:** Finds a client by their name in the database.
- **Parameters:**
 - **String name:** The name of the client to find.
- **Returns:** The **Clients** object representing the found client, or **null** if not found.

Class: OrderDAO

Description: The OrderDAO class provides methods for managing database operations related to orders, including inserting new orders into the database.

Fields:

- None explicitly declared in this class.

Constructor:

- **public OrderDAO():** Constructs a new OrderDAO instance. Inherits from the AbstractDAO class.

Methods:**insert**

- **Description:** Inserts a new order into the database.
- **Parameters:**
 - **Orders order:** The Orders object representing the new order to insert.
- **Returns:** The inserted order object with updated ID if applicable, or null on failure.

Class: OrderProductsDAO

Description: The OrderProductsDAO class provides methods for managing database operations related to order products, specifically inserting new order products into the database.

Fields:

- None explicitly declared in this class.

Constructor:

- **public OrderProductsDAO():** Constructs a new OrderProductsDAO instance. Inherits from the AbstractDAO class.

Methods:**insert**

- **Description:** Inserts a new order product into the database.
- **Parameters:**
 - **OrderProducts orderProducts:** The OrderProducts object representing the new order product to insert.
- **Returns:** The inserted order product object with updated ID if applicable, or null on failure.

Class: ProductDAO

Description: The ProductDAO class provides methods for managing database operations related to products, such as finding, inserting, updating, and deleting products in the database.

Fields:

- None explicitly declared in this class.

Constructor:

- **public ProductDAO():** Constructs a new ProductDAO instance. Inherits from the AbstractDAO class.

Methods:

findById

- **Description:** Finds a product by its ID.
- **Parameters:**
 - **int productId:** The ID of the product to find.
- **Returns:** The Products object representing the found product, or null if not found.

insert

- **Description:** Inserts a new product into the database.
- **Parameters:**
 - **Products newProduct:** The Products object representing the new product to insert.
- **Returns:** The inserted product object with updated ID if applicable, or null on failure.

update

- **Description:** Updates an existing product in the database.
- **Parameters:**
 - **Products productWithUpdatedValues:** The Products object representing the updated product values.
- **Returns:** The updated product object, or null on failure.

delete

- **Description:** Deletes a product from the database by its ID.
- **Parameters:**
 - **int id:** The ID of the product to delete.
- **Returns:** The deleted product object, or null on failure.

findAll

- **Description:** Retrieves a list of all products from the database.
- **Returns:** A list of Products objects representing all products found.

Class: ConnectionFactory

Description: The `ConnectionFactory` class serves as a factory for managing database connections and resources such as statements and result sets.

Fields:

- **private static final String DRIVER:** The JDBC driver class name for PostgreSQL.
- **private static final String DBURL:** The URL to connect to the PostgreSQL database.
- **private static final String USER:** The username for database authentication.
- **private static final String PASS:** The password for database authentication.

Constructor:

- **private ConnectionFactory():** Private constructor used for initializing the JDBC driver.

Methods:

createConnection

- **Description:** Creates and returns a connection to the PostgreSQL database.
- **Returns:** A `Connection` object representing the database connection.

getConnection

- **Description:** Retrieves a connection to the database using the singleton `singleInstance` of `ConnectionFactory`.
- **Returns:** A `Connection` object representing the database connection.

close(Connection connection)

- **Description:** Closes the provided database connection.
- **Parameters:**
 - **connection:** The `Connection` object to close.

close(Statement statement)

- **Description:** Closes the provided database statement.
- **Parameters:**
 - **statement:** The `Statement` object to close.

close(ResultSet resultSet)

- **Description:** Closes the provided database result set.
- **Parameters:**
 - **resultSet:** The `ResultSet` object to close.

Class: AmountValidator

Description: The `AmountValidator` class provides utility methods for validating the amount of products.

Methods:

validate

- **Description:** Validates the amount for a product, ensuring it is not negative or zero.
- **Parameters:**
 - **t:** The `Products` object whose amount needs to be validated.
- **Throws:**

- **IllegalArgumentException:** If the amount is negative or zero.

Class: EmailValidator

Description: The EmailValidator class provides utility methods for validating email addresses.

Fields:

- **EMAIL_PATTERN:** Regular expression pattern for validating email addresses, ensuring it contains @ and .com.

Methods:

validate

- **Description:** Validates the email address of a client.
- **Parameters:**
 - **t:** The Clients object whose email address needs to be validated.
- **Throws:**
 - **IllegalArgumentException:** If the email address is not valid based on the defined pattern.

Class: ClientBLL

Description: The ClientBLL class represents the Business Logic Layer responsible for managing clients. It handles operations such as retrieving clients, inserting new clients, updating existing clients, and deleting clients.

Fields:

- **clientDAO:** Instance of the ClientDAO class for interacting with client data in the database.
- **emailValidator:** Instance of the EmailValidator class for validating email addresses associated with clients.

Methods:

findClientById

- **Description:** Retrieves a client by its ID.
- **Parameters:**
 - **clientId:** The ID of the client to retrieve.
- **Returns:** The client with the specified ID.
- **Throws:**
 - **NoSuchElementException:** If the client with the specified ID is not found.

insertNewClient

- **Description:** Inserts a new client after validating its email address.
- **Parameters:**
 - **newClient:** The client to insert.

- **Returns:** The inserted client.

updateClient

- **Description:** Updates an existing client after validating its email address.
- **Parameters:**
 - **clientWithUpdatedValues:** The updated client object.
- **Returns:** The updated client.

deleteClient

- **Description:** Deletes a client by its ID.
- **Parameters:**
 - **id:** The ID of the client to delete.
- **Returns:** The deleted client.

findAllClients

- **Description:** Retrieves all clients.
- **Returns:** A list of all clients.

findByName

- **Description:** Finds a client by its name.
- **Parameters:**
 - **name:** The name of the client to find.
- **Returns:** The client with the specified name.

Class: OrderBLL

Description: The OrderBLL class represents the Business Logic Layer responsible for managing orders. It provides methods for creating new orders and interacts with the OrderDAO class to handle database operations related to orders.

Fields:

- **orderDAO:** Instance of the OrderDAO class for interacting with order data in the database.

Methods:

createOrder

- **Description:** Creates a new order for a client with the specified date.
- **Parameters:**
 - **clientId:** The ID of the client placing the order.
 - **date:** The timestamp representing the order date.
- **Returns:** The created order object.

Class: ProductBLL

Description: The ProductBLL class represents the Business Logic Layer responsible for managing products. It provides methods for CRUD operations on products and validates product amounts using the AmountValidator class.

Fields:

- **productDAO:** Instance of the ProductDAO class for interacting with product data in the database.
- **amountValidator:** Instance of the AmountValidator class for validating product amounts.

Methods:

findProductById

- **Description:** Finds a product by its ID.
- **Parameters:**
 - **productId:** The ID of the product to find.
- **Returns:** The product with the specified ID.

insertNewProduct

- **Description:** Inserts a new product after validating its amount.
- **Parameters:**
 - **newProduct:** The product to insert.
- **Returns:** The inserted product.

updateProduct

- **Description:** Updates an existing product after validating its amount.
- **Parameters:**
 - **productWithUpdatedValues:** The updated product object.
- **Returns:** The updated product.

deleteProduct

- **Description:** Deletes a product by its ID.
- **Parameters:**
 - **id:** The ID of the product to delete.
- **Returns:** The deleted product.

findAllProducts

- **Description:** Retrieves all products.
- **Returns:** A list of all products.

5. Conclusions

What have I learned:

- The 4 layers architectural pattern
- To build interfaces (GUI) using Java Swing
- Reflection technique
- Generic classes
- Working with a database from a Java application

Future developments:

- GUI improvements
- Adding more possible operations

6. Bibliography

7. *PT Course from TUCN* - <https://dsrl.eu/courses/pt/>